Function

Introduction....4
Function Basics.... 5
Function Declaration in C....5
Function Definition in C....6
Function Call in C....7
Syntax....7
Example....7
Types of Functions in C.... 7
1. Library Functions.... 8
2. User Defined Functions.... 9
Steps of function executions in program....10
Types of User Defined Functions.... 11
1. Function with Arguments and Return Value.... 11
2. Function with Arguments and No Return Value.... 12
3. Function with No Arguments and Return Value.... 14
4. Function with No Arguments and No Return Value....15
Different types of passing argument to function.... 17
1) Call by Value....17
2) Call by Reference.... 18
Recursion in Function.... 19
Program.... 20
Types of Recursion.... 22
1. Direct Recursion.... 22
2. Indirect Recursion....23
3. Tail Recursion.... 24
4. Non-Tail Recursion....24
5. Nested Recursion.... 25
Variable Argument Functions in C (printf() and scanf())....26
User-Defined Variable Argument Function....27
Practice Questions.... 29
Easy Level (Functions Basics)....29
Medium Level.... 29
Hard Level (Advanced + Recursion + Pointers).... 30

Introduction

A function is a block of statements designed to perform a specific task. It may take input values called **arguments**, process them, and return an output called a **return value**.

A C program is made up of one or more functions. Therefore, a function is also called a **subprogram**, **routine**, or **method** in different contexts.

Every C program must contain at least one function: main()

The main() function is the starting point of program execution, so it is called the **entry point function**.

Using multiple functions helps in:

- Modular programming
- Better readability
- Easier debugging and maintenance
- Code reusability
- Faster development for large projects

In large programs, tasks are divided into separate functions, making the code organized and easier to manage. C also provides many **built-in library functions**, and programmers can create their own **user-defined functions**.

Function Basics

To understand functions in C, three fundamental terminologies are important:

1. **Function Declaration**
   Also called a **function prototype** or **signature**.It tells the compiler about the function name, return type, and parameters.
2. **Function Definition**
   Also called the **implementation** or **function body**. It contains the actual code that performs the task.
3. **Function Call**
   Used to execute the function from another part of the program.

Function Declaration in C

A **function declaration** (also called a **function prototype**) informs the compiler about:

- Function name
- Return type

- Arguments (parameters)

It is usually written at the top of the program or before the function is called.

Syntax

```
return_type function_name(data_type argument1, data_type argument2);
```

Example

```
int fact(int n);
```

This declaration tells the compiler:

- fact is the function name
- It accepts one integer argument
- It returns an integer value

Important Points

- If a function does not return any value, use void as the return type.--> void print(void);
- Function arguments must include their data types.
- Multiple arguments are separated by commas.
- A function declaration always ends with a semicolon (;).

Examples of Function Declarations

```
int fact(int n);          // Returns int, accepts int
void print(char ch);      // No return value, accepts char
float sum(float a, float b); // Returns float, accepts two floats
void task(void);          // No arguments and no return value
```

Function Definition in C

A **function definition** contains the actual set of statements that perform a specific task.
These statements are written inside the function body.

Syntax

```
return_type function_name(data_type arg1, data_type arg2, ...)
{
   // set of statements
}
```

Example

```
int add(int a, int b)
{
   return a + b;
}
```

Important Points

- The function body is enclosed within { }.
- It contains executable statements.
- A function may accept arguments and may return a value.
- The return type specifies the type of value returned by the function.

Function Call in C

A **function call** is used to execute a function from another function, usually from main().

The function definition contains the actual code, while the function declaration informs the compiler about the function before it is defined.

Syntax

```
variable_name = function_name(arg1, arg2, ...);
```

Example

```
result = fact(num);
```

Here:

- fact → function name
- num → actual argument
- result → variable used to store the return value

Important Points

- A function is called using its name followed by parentheses ().
- Values passed during the function call are called **actual arguments**.
- Parameters written in the function definition are called **formal arguments**.
- Actual argument values are copied into formal arguments.
- After execution, control returns to the statement following the function call.
- If a function returns a value, it can be stored in a variable.

- Some functions may ignore the return value, such as printf().

Example

```
printf("Hello");
```

Here, the return value of printf() is not stored.

Types of Functions in C

Functions in C are mainly divided into **two types**:

1. **Library Functions**
2. **User Defined Functions**

1. Library Functions

- C provides a set of predefined (library) functions.
- These functions are:
  - Declared in header files (.h) like stdio.h, string.h, etc.
  - Defined in library files (.lib or .so).
- These libraries are linked with the program during compilation.

main() Function

- main() is a user-defined function, but it has special importance.
- Program execution always starts from main().
- The operating system calls main() when the program runs.

Standard Declaration

```
int main(int argc, char *argv[]);
```

- Used when arguments are passed from the command line.

Other Valid Forms

int main(void);

int main();

Return Type of main()

- According to ANSI standard:
  - The return type of main() must be int.

- The return value is passed back to the operating system.

Common Return Values

- 0 → Program executed successfully
- 1 → Program terminated due to error (e.g., lack of resources)
- -1 → Abnormal termination

Non-Standard Forms

void main();

void main(int argc, char *argv[]);

- These are not recommended (non-standard).
- In such cases, return value is assumed as 0.

Nested Function Calls

- In C:
  - Function definitions cannot be nested
  - But function calls can be nested

Example

printf("%d", printf("technoscript"));

Explanation

1. Inner printf("technoscript") executes first
2. It prints: technoscript
3. It returns the number of characters printed → 12
4. Outer printf prints: 12

Final Output : technoscript12

2. User Defined Functions

These are functions created by the programmer according to requirement.

Syntax

```
return_type function_name(parameters)
{
   // function body
```

```
}
```

Example Program

```
#include <stdio.h>

void display()
{
   printf("Hello World");
}

int main()
{
   display();

   return 0;
}
```

Output
Hello World

Parts of User Defined Function

Part	Description
Function Declaration	Tells compiler about function
Function Definition	Contains actual code
Function Call	Executes the function

Program

```
#include <stdio.h>

int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}

int main()
```

```
{
    int x = 10, y = 20, result;
    result = add(x, y);
    printf("Sum = %d", result);
}
```

Steps of function executions in program

1. **Program starts → main() executes**
    - Stack frame for main() is created
2. **Variables initialized in main()**
    - x = 10, y = 20, result = ?
3. **Function call → add(x, y)**
    - Values passed (call by value)
    - New stack frame for add() is created
4. **Execution inside add()**
    - sum = a + b = 30
5. **Return statement**
    - Value 30 returned to main()
6. **Stack frame removed**
    - add() frame popped from stack
7. **Back to main()**
    - result = 30
8. **Output printed**
    - Sum = 30
9. **Program ends**
    - Stack becomes empty.

Types of User Defined Functions

1. Function with Arguments and Return Value

Definition

- Data is passed from main() to the function.
- Function processes the data.
- Function returns a result back to main().

Program: Addition of Two Numbers

```c
#include <stdio.h>

int add(int a, int b);

int main()
{
   int result;

   result = add(10,20);

   printf("Sum = %d", result);

   return 0;
}

int add(int a, int b)
{
   return a+b;
}
```

Step-by-Step Execution

Step 1

Function call: result = add(10,20);

Values passed: a = 10 , b = 20

Step 2

Control goes to → int add(int a,int b)

Step 3

Calculate: a+b

10+20 = 30

Step 4

Return: return 30;

Step 5

Value stored: result = 30;

Step 6

Print: Sum = 30

2. Function with Arguments and No Return Value

Definition

- Data is passed to the function.
- Function performs the task.
- Function does not return any value.
- Return type is void.

Program: Addition of Two Numbers

```c
#include <stdio.h>

void add(int a, int b);

int main()
{
   add(10,20);

   return 0;
}

void add(int a, int b)
{
   int sum;

   sum = a+b;

   printf("Sum = %d", sum);
}
```

Step-by-Step Execution

Step 1

Call: add(10,20);

Step 2

Values received: a = 10, b = 20

Step 3

Calculate: sum = a+b;

sum = 30;

Step 4

Print: Sum = 30

Step 5

Control returns automatically to main().

No value is returned.

3. Function with No Arguments and Return Value

Definition

- No data is passed from main().
- Function itself collects or creates data.
- Function returns a value.

Program: Addition of Two Numbers

```
#include <stdio.h>

int add(void);

int main()
{
   int result;

   result = add();

   printf("Sum = %d", result);
```

```
    return 0;
}

int add(void)
{
    int a,b;

    printf("Enter Two Numbers: ");
    scanf("%d%d",&a,&b);

    return a+b;
}
```

Step-by-Step Execution

Step 1

Call: add();

No arguments passed.

Step 2

Function asks for input: 10 20

Step 3

Calculate: 10 + 20 = 30

Step 4

Return: return 30;

Step 5

Store: result = 30;

Step 6

Print: Sum = 30

4. Function with No Arguments and No Return Value

Definition

- No data passed from main().
- No value returned to main().
- Function performs the complete task itself.

Program: Addition of Two Numbers

```c
#include <stdio.h>

void add(void);

int main()
{
   add();

   return 0;
}

void add(void)
{
   int a,b,sum;

   printf("Enter Two Numbers: ");
   scanf("%d%d",&a,&b);

   sum = a+b;

   printf("Sum = %d", sum);
}
```

Step-by-Step Execution

Step 1

Call: add();

Step 2

Function receives no arguments.

Step 3

Take input: 10 20

Step 4

Calculate: sum = 30;

Step 5

Print: Sum = 30

Step 6

Function finishes. No value returned.

Comparison of All Four Types

Function Type	Arguments	Return Value
With argument, with return	Yes	Yes
With argument, no return	Yes	No
No argument, with return	No	Yes
No argument, no return	No	No

Different types of passing argument to function

1) Call by Value

- In **call by value**, a **copy of the actual argument** is passed to the function.
- Changes made inside the function **do not affect** the original variable.

Key Points

- Memory is **separate** for actual and formal parameters
- Safe (original data is not modified)
- Default method in C

Example

```c
#include <stdio.h>

void modify(int x)
{
   x = x + 10;
   printf("Inside function: %d\n", x);
}

int main()
{
   int a = 5;
   modify(a);
   printf("Outside function: %d\n", a);
   return 0;
}
```

Output

Inside function: 15
Outside function: 5

Conclusion

- Value of a remains unchanged → **Call by Value**

2) Call by Reference

- In **call by reference**, the **address of the variable** is passed.
- Changes inside the function **affect the original variable**.

Key Points

- Uses **pointers**
- Same memory is accessed
- Allows modification of original data

Example

```c
#include <stdio.h>
```

```c
void modify(int *x)
{
   *x = *x + 10;
   printf("Inside function: %d\n", *x);
}

int main()
{
   int a = 5;
   modify(&a);
   printf("Outside function: %d\n", a);
   return 0;
}
```

Output

Inside function: 15
Outside function: 15

Conclusion

- Value of a changes → **Call by Reference**

Difference Between Call by Value & Call by Reference

Feature	Call by Value	Call by Reference
What is passed	Copy of value	Address of variable
Memory	Separate	Same
Effect on original	No change	Changes original
Safety	More safe	Less safe (direct access)
Use	When no modification needed	When modification required

Important Note

- C **does not support true call by reference directly**
- It is implemented using **pointers (call by address)**

Recursion in Function

- Recursion is a technique where a function calls itself.
- It is used to solve problems that can be broken into smaller sub-problems.

Basic Structure of Recursion

Every recursive function must have:

1. Base Case → Condition to stop recursion
2. Recursive Case → Function calls itself

General Form

return_type function_name(parameters)

{

if(base_condition)

return value;

else

return function_name(smaller_problem);

}

Example 1: Factorial (Classic Recursion)

Logic

- Factorial of n: n! = n × (n-1)!
- Base case: 0! = 1

Program

```c
#include <stdio.h>

int fact(int n)
{
  if(n == 0)
     return 1;         // Base case
  else
     return n * fact(n - 1);  // Recursive call
```

```
}

int main()
{
   int n = 5;
   printf("Factorial = %d", fact(n));
   return 0;
}
```

Step-by-Step Execution (n = 5)

fact(5)

= 5 * fact(4)

= 5 * 4 * fact(3)

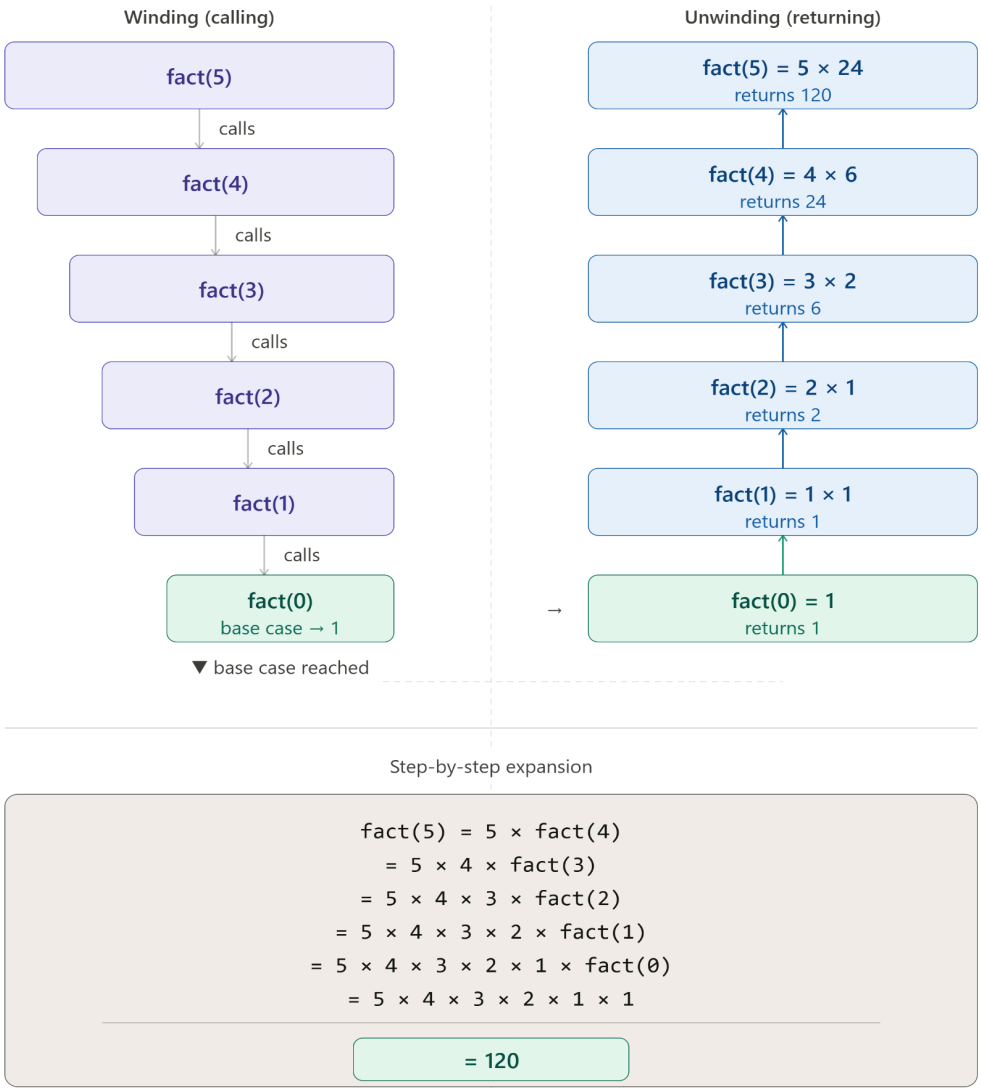
= 5 * 4 * 3 * fact(2)

= 5 * 4 * 3 * 2 * fact(1)

= 5 * 4 * 3 * 2 * 1 * fact(0)

= 5 * 4 * 3 * 2 * 1 * 1

= 120



Example 2: Fibonacci Series

Logic : f(n) = f(n-1) + f(n-2)

Program

```
#include <stdio.h>

int fib(int n)
{
  if(n == 0)
```

```
    return 0;
  else if(n == 1)
    return 1;
  else
    return fib(n-1) + fib(n-2);
}

int main()
{
  int i;
  for(i = 0; i < 6; i++)
    printf("%d ", fib(i));
  return 0;
}
```

Output

0 1 1 2 3 5

Types of Recursion

1. Direct Recursion

- **Function calls itself directly**

Example

```
#include <stdio.h>

void display(int n)
{
  if(n == 0)
    return;
  printf("%d ", n);
  display(n - 1);
}

int main()
{
  display(5);
  return 0;
}
```

Output:

5 4 3 2 1

2. Indirect Recursion

- Function calls another function, which calls the first function.

Example

```c
#include <stdio.h>

void fun2(int);

void fun1(int n)
{
  if(n > 0)
  {
     printf("%d ", n);
     fun2(n - 1);
  }
}

void fun2(int n)
{
  if(n > 0)
  {
     printf("%d ", n);
     fun1(n - 1);
  }
}

int main()
{
  fun1(5);
  return 0;
}
```

3. Tail Recursion

- **Recursive call is the last statement**

Example

```c
#include <stdio.h>

void print(int n)
{
   if(n == 0)
       return;
   printf("%d ", n);
   print(n - 1);   // Last statement
}

int main()
{
   print(5);
   return 0;
}
```

4. Non-Tail Recursion

- **Recursive call is not the last statement**

Example

```c
#include <stdio.h>

void print(int n)
{
   if(n == 0)
       return;
   print(n - 1);
   printf("%d ", n);
}

int main()
{
   print(5);
   return 0;
}
```

Output:

1 2 3 4 5

5. Nested Recursion

- **Function call is inside another recursive call**

Example

```c
#include <stdio.h>

int fun(int n)
{
   if(n > 100)
      return n - 10;
   else
      return fun(fun(n + 11));
}

int main()
{
   printf("%d", fun(95));
   return 0;
}
```

Advantages of Recursion

- Makes code short and simple
- Useful for tree, graph, factorial, Fibonacci problems

Disadvantages

- Uses more memory (stack)
- Can be slow due to repeated calls
- Risk of infinite recursion if base case is missing

Variable Argument Functions in C (printf() and scanf())

When teaching user-defined functions, it is useful to mention that some library functions like printf() and scanf() can accept a variable number of arguments.

What is a Variable Argument Function?

A Variable Argument Function is a function that can accept different numbers of arguments during different function calls.

Example: printf("Hello");

One argument.

printf("Age = %d", age);

Two arguments.

printf("A=%d B=%d C=%d", a, b, c);

Four arguments.

The same function printf() accepts different numbers of arguments.

Examples

printf()

printf("Hello");

printf("Value = %d", x);

printf("%d %d %d", a, b, c);

scanf()

scanf("%d", &a);

scanf("%d %d", &a, &b);

scanf("%d %f %c", &a, &f, &ch);

Why Are They Called Variable Argument Functions?

Because the number of arguments is not fixed.

For example:

printf("Hello");

1 argument.

printf("%d", x);

2 arguments.

printf("%d %d %d", a, b, c);

4 arguments.

The compiler allows all of them.

Internal Working

The prototype of printf() is simplified as:

int printf(const char *format, ...);

Notice:

...

These three dots are called ellipsis.

They indicate that the function can receive additional arguments.

Similarly:

int scanf(const char *format, ...);

also accepts variable arguments.

User-Defined Variable Argument Function

C provides the header file: **#include <stdarg.h>**

which allows programmers to create their own variable argument functions.

Example:

```
#include <stdio.h>
#include <stdarg.h>

int sum(int n, ...)
{
   va_list args;
   int i, total = 0;

   va_start(args, n);
```

```
    for(i = 0; i < n; i++)
    {
        total += va_arg(args, int);
    }

    va_end(args);

    return total;
}

int main()
{
    printf("%d\n", sum(3, 10, 20, 30));
    printf("%d\n", sum(5, 1, 2, 3, 4, 5));

    return 0;
}
```

Output:

60

15

Comparison with Normal Functions

Normal Function

int add(int a, int b)

{

  return a+b;

}

Only two arguments allowed.

Valid: add(10,20);

Invalid: add(10,20,30);

Variable Argument Function

int sum(int n, ...)

Valid:
sum(2,10,20);

sum(3,10,20,30);

sum(5,1,2,3,4,5);

Number of arguments can vary.

Practice Questions

Easy Level (Functions Basics)

1. Write a C program to create a function add() that accepts two integers and returns their sum. Call the function from main().
2. Write a C program to create a function isEven() that checks whether a number is even or odd and returns the result.
3. Write a C program to find the maximum of two numbers using a user-defined function.
4. Write a C program to calculate the square of a number using a function with return value.
5. Write a C program to print the multiplication table of a number using a function (no return value).
6. Write a C program to find factorial of a number using a function (non-recursive).

Medium Level

7. Write a C program to check whether a number is prime using a function.
8. Write a C program to reverse a number using a function.
9. Write a C program to find sum of digits of a number using a function.
10. Write a C program to swap two numbers:
- (a) using call by value
- (b) using call by reference
11. Write a C program to generate Fibonacci series using a function.
12. Write a C program to calculate power xnx^nxn using a function without using library functions.

Hard Level (Advanced + Recursion + Pointers)

13. Write a C program to find factorial of a number using recursion.
14. Write a C program to find sum of digits of a number using recursion.
15. Write a C program to find GCD (Greatest Common Divisor) of two numbers using recursion.
16. Write a C program to pass an array to a function and find the largest element.
17. Write a C program to create a function that uses a static variable to count how many times the function is called.
18. Write a C program to return a pointer from a function that points to the maximum element of an array.
19. Write a C program to check whether a number is palindrome using a function.
20. Write a C program to demonstrate nested function calls and trace the output.